

Guía Profesional: Trabajo en Equipo con Git

Introducción

Git es la herramienta de control de versiones más utilizada en la industria del software. Esta guía te proporcionará las mejores prácticas y flujos de trabajo para colaborar efectivamente con tu equipo.

Configuración Inicial

Configuración de Identidad

```
bash

git config --global user.name "Tu Nombre"
git config --global user.email "tu.email@empresa.com"
```

Configuración Recomendada

```
bash

# Editor predeterminado
git config --global core.editor "code --wait"

# Evitar commits accidentales en main/master
git config --global init.defaultBranch main

# Mejor visualización de logs
git config --global alias.lg "log --graph --pretty=format:%Cred%H%Creset -%C(yellow)%d%Creset %s %Cgreen(%cr) %C(b
```

Estrategia de Branching: Git Flow Simplificado

Ramas Principales

- main/master:** Código en producción, siempre estable.
- develop:** Rama de integración donde converge el trabajo del equipo.

Ramas de Trabajo

feature/: Para nuevas funcionalidades

```
bash

feature/login-oauth
feature/user-dashboard
feature/payment-integration
```

bugfix/: Para corrección de bugs en develop

```
bash
```

```
bugfix/incorrect-validation
```

```
bugfix/memory-leak
```

hotfix/: Para correcciones urgentes en producción

```
bash
```

```
hotfix/security-patch
```

```
hotfix/critical-payment-error
```

Flujo de Trabajo Diario

1. Comenzar una Nueva Tarea

```
bash
```

```
# Actualizar develop
```

```
git checkout develop
```

```
git pull origin develop
```

```
# Crear rama de feature
```

```
git checkout -b feature/nombre-descriptivo
```

```
# Ejemplo:
```

```
git checkout -b feature/add-password-reset
```

2. Trabajar en tu Rama

```
bash
```

```
# Hacer cambios en el código
```

```
# Ver qué archivos cambiaron
```

```
git status
```

```
# Agregar archivos al staging
```

```
git add archivo.js
```

```
# o todos los cambios
```

```
git add .
```

```
# Hacer commit con mensaje descriptivo
```

```
git commit -m "feat: implement password reset functionality"
```

3. Mantener tu Rama Actualizada

```
bash
```

```
# Traer cambios de develop periódicamente
```

```
git checkout develop
```

```
git pull origin develop
```

```
git checkout feature/tu-rama
```

```
git rebase develop
```

```
# Si hay conflictos, resuélvelos y continúa
```

```
git add archivo-resuelto.js
```

```
git rebase --continue
```

4. Subir Cambios al Repositorio Remoto

```
bash
```

```
# Primera vez
```

```
git push -u origin feature/tu-rama
```

```
# Subsiguientes pushes
```

```
git push
```

```
# Si hiciste rebase, necesitas forzar (con precaución)
```

```
git push --force-with-lease
```

5. Crear Pull Request (PR)

Desde GitHub/GitLab/Bitbucket:

- Ve a tu repositorio

- Selecciona tu rama
- Click en "New Pull Request" o "Create Merge Request"
- Completa descripción, asigna revisores
- Vincula issues relacionados

Convenciones de Commits (Conventional Commits)

Formato

<tipo>(<alcance>): <descripción>

[cuerpo opcional]

[footer opcional]

Tipos Principales

- **feat**: Nueva funcionalidad
- **fix**: Corrección de bug
- **docs**: Cambios en documentación
- **style**: Cambios de formato (espacios, indentación)
- **refactor**: Refactorización sin cambiar funcionalidad
- **test**: Agregar o modificar tests
- **chore**: Tareas de mantenimiento

Ejemplos

bash

```
git commit -m "feat(auth): add OAuth2 login with Google"
```

```
git commit -m "fix(api): resolve null pointer in user endpoint"
```

```
git commit -m "docs(readme): update installation instructions"
```

```
git commit -m "refactor(utils): simplify date formatting function"
```

```
git commit -m "test(auth): add unit tests for login validation"
```

Code Review: Mejores Prácticas

Como Autor del PR

1. **Mantén los PRs pequeños**: Idealmente < 400 líneas de cambio

2. **Descripción clara:** Explica qué, por qué y cómo
3. **Auto-revisión:** Revisa tu propio código antes de solicitar revisión
4. **Tests incluidos:** Asegúrate que los tests pasen
5. **Responde constructivamente:** Acepta feedback con apertura

Como Revisor

1. **Revisa con prontitud:** Idealmente dentro de 24 horas
2. **Sé específico y constructivo:** No digas "esto está mal", explica por qué
3. **Pregunta en lugar de ordenar:** "¿Consideraste usar X?" vs "Usa X"
4. **Aprueba cuando esté listo:** No retengas aprobación por preferencias personales
5. **Reconoce el buen trabajo:** Un "¡Excelente implementación!" motiva

Resolución de Conflictos

Proceso

bash

```
# Actualizar develop
```

```
git checkout develop
```

```
git pull origin develop
```

```
# Volver a tu rama
```

```
git checkout feature/tu-rama
```

```
# Rebase para traer cambios
```

```
git rebase develop
```

```
# Git te indicará los archivos con conflicto
```

```
# Abre cada archivo y busca los marcadores:
```

```
<<<<<<< HEAD
```

```
// Tu código
```

```
=====
```

```
// Código de develop
```

```
>>>>>>> develop
```

```
# Decide qué código mantener o combina ambos
```

```
# Una vez resuelto
```

```
git add archivo-resuelto.js
```

```
git rebase --continue
```

```
# Si hay más conflictos, repite el proceso
```

```
# Si necesitas abortar
```

```
git rebase --abort
```

Herramientas Visuales

```
bash
```

```
# Configurar herramienta de merge
```

```
git config --global merge.tool vscode
```

```
git config --global mergetool.vscode.cmd "code --wait $MERGED"
```

```
# Usar herramienta visual
```

```
git mergetool
```

Comandos Útiles para el Día a Día

Exploración e Información

```
bash
```

Ver historial visual

```
git log --graph --oneline --all
```

Ver cambios no commiteados

```
git diff
```

Ver cambios en staging

```
git diff --staged
```

Ver quién modificó cada línea

```
git blame archivo.js
```

Buscar en el historial

```
git log --grep="password"
```

Deshacer Cambios

bash

Descartar cambios locales (sin commit)

```
git checkout -- archivo.js
```

Deshacer último commit (mantiene cambios)

```
git reset --soft HEAD~1
```

Deshacer último commit (descarta cambios)

```
git reset --hard HEAD~1
```

Crear commit que revierta otro commit

```
git revert <hash-del-commit>
```

Gestión de Stash (Cambios Temporales)

bash

```
# Guardar cambios sin commit
```

```
git stash
```

```
# Guardar con mensaje descriptivo
```

```
git stash save "WIP: working on login form"
```

```
# Ver lista de stashes
```

```
git stash list
```

```
# Aplicar último stash
```

```
git stash pop
```

```
# Aplicar stash específico
```

```
git stash apply stash@{1}
```

```
# Eliminar stash
```

```
git stash drop stash@{0}
```

Errores Comunes y Soluciones

"Olvidé cambiarme de rama antes de hacer cambios"

```
bash
```

```
# Si NO hiciste commit
```

```
git stash
```

```
git checkout rama-correcta
```

```
git stash pop
```

```
# Si ya hiciste commit
```

```
git log # copiar hash del commit
```

```
git checkout rama-correcta
```

```
git cherry-pick <hash>
```

"Hice commit en la rama equivocada"

```
bash
```

```
# Copiar hash del commit
```

```
git log
```

```
# Ir a rama correcta
```

```
git checkout rama-correcta
```

```
git cherry-pick <hash>
```

```
# Volver a rama incorrecta y deshacer
```

```
git checkout rama-incorrec
```

```
git reset --hard HEAD~1
```

"Necesito cambiar el mensaje del último commit"

```
bash
```

```
git commit --amend -m "nuevo mensaje"
```

"Empujé código que no debía subir"

```
bash
```

```
# CUIDADO: Solo si nadie más ha bajado esos cambios
```

```
git reset --hard HEAD~1
```

```
git push --force-with-lease
```

Reglas de Oro del Trabajo en Equipo

1. **Nunca fuerces push en main/develop:** Usa `--force-with-lease` solo en tus ramas de feature
2. **Pull antes de push:** Siempre actualiza antes de subir cambios
3. **Commits atómicos:** Cada commit debe representar un cambio lógico completo
4. **No commitees código que no compila:** Tu código debe pasar tests antes de push
5. **Comunica cambios breaking:** Si tu código rompe compatibilidad, avisa al equipo
6. **Documenta decisiones importantes:** Usa el cuerpo del commit para explicar el "por qué"

Workflow Completo: Ejemplo Práctico

```
bash
```

Día 1: Comenzar feature

`git checkout develop`

`git pull origin develop`

`git checkout -b feature/user-profile-page`

... desarrollar ...

`git add .`

`git commit -m "feat(profile): create user profile page structure"`

`git push -u origin feature/user-profile-page`

Día 2: Continuar trabajando

`git pull origin develop` *# actualizar con cambios del equipo*

`git rebase develop` *# integrar cambios*

... desarrollar más ...

`git add .`

`git commit -m "feat(profile): add avatar upload functionality"`

`git push`

Día 3: Finalizar y solicitar revisión

`git add .`

`git commit -m "test(profile): add unit tests for profile page"`

`git push`

Crear Pull Request en la plataforma

Esperar code review y hacer ajustes si es necesario

Después de aprobación

El PR se mergea a develop

`git checkout develop`

`git pull origin develop`

`git branch -d feature/user-profile-page` *# eliminar rama local*

Herramientas y Recursos

GUIs Recomendadas

- **GitKraken:** Visual y potente
- **SourceTree:** Gratuita de Atlassian
- **GitHub Desktop:** Simple para principiantes
- **VS Code Git Integration:** Integrada en el editor

Recursos de Aprendizaje

- [Learn Git Branching](#) - Interactivo
- [Oh Shit, Git!?!](#) - Soluciones rápidas

- Git Documentation - Documentación oficial

Checklist Pre-Push

Antes de cada push, verifica:

- ☐ Mi código compila sin errores
- ☐ Los tests pasan (si existen)
- ☐ No hay conflictos con develop
- ☐ El mensaje de commit sigue las convenciones
- ☐ No incluyo archivos sensibles (.env, credenciales)
- ☐ El código está formateado correctamente
- ☐ He revisado los cambios con `git diff`

Conclusión

Git es una herramienta poderosa que requiere práctica. No temas experimentar en ramas locales, siempre puedes volver atrás. La clave del trabajo en equipo exitoso es la comunicación constante y el respeto por el código compartido.

Recuerda: **commit temprano, commit frecuentemente, y mantén tu rama actualizada.**

Documento creado por: Tech Lead / DevOps Senior

Última actualización: Enero 2026

Feedback: Siempre bienvenido para mejorar este documento